



HỆ ĐIỀU HÀNH

CHƯƠNG 4: ĐỊNH THỜI CPU (PHẦN 1)

Định thời CPU là hoạt động quan trọng của thành phần quản lý tiến trình và có ảnh hưởng rất lớn đến hiệu suất máy tính cũng như trải nghiệm của người dùng. Trong chương này, người học được trình bày về mục đích và các tiêu chuẩn định thời, cũng như các chiến lược định thời CPU cơ bản.



MỤC TIÊU

1. Biết được các khái niệm cơ bản về định thời
2. Biết được các tiêu chuẩn định thời CPU
3. Hiểu được các giải thuật định thời
4. Vận dụng các giải thuật định thời để làm bài tập và mô phỏng



NỘI DUNG

1. Các khái niệm cơ bản về định thời
2. Các loại định thời
3. Các tiêu chuẩn định thời CPU
4. Các giải thuật định thời
 - First-Come, First-Served (FCFS)



CÁC KHÁI NIỆM CƠ BẢN VỀ ĐỊNH THỜI

01.



1. Các khái niệm cơ bản về định thời

- Trong các hệ thống đa nhiệm (multitasking), đơn vị xử lý
 - Cho phép thực thi đồng thời nhiều chương trình để làm tăng hiệu suất hệ thống (*Cho phép nhiều chương trình được nạp vào bộ nhớ*).
 - Tại mỗi thời điểm, chỉ có một tiến trình được thực thi.
- Cần phải giải quyết vấn đề phân chia, lựa chọn tiến trình thực thi để đạt được hiệu quả cao nhất.
- Cần có những phương pháp chọn lựa phù hợp.

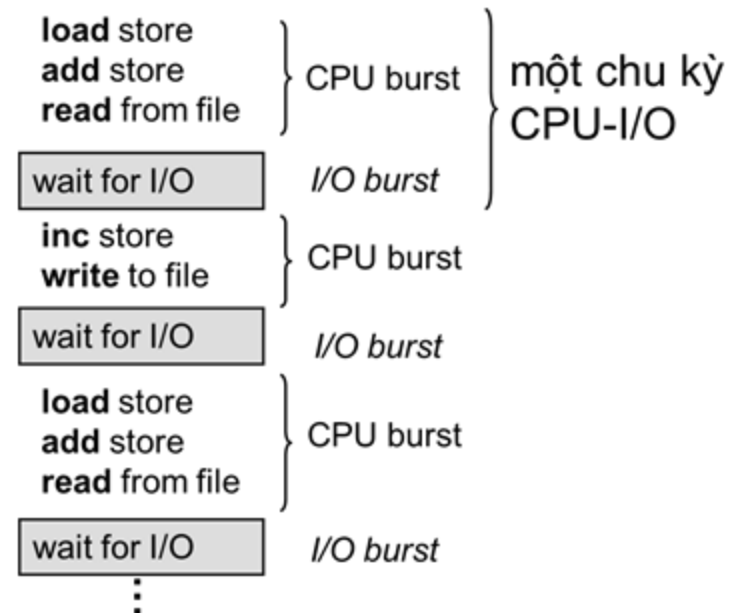
Định thời là chiến lược lựa chọn tiến trình phù hợp để được thực thi sao cho đạt được hiệu quả cao nhất.



1. Các khái niệm cơ bản về định thời

Chu kỳ CPU-I/O

- **Service time** là thời gian một tiến trình cần CPU trong một chu kỳ CPU - I/O (hay còn gọi là **burst time**).
- Tiến trình có *service time* lớn được gọi là các **tiến trình hướng CPU** (CPU-bound process).



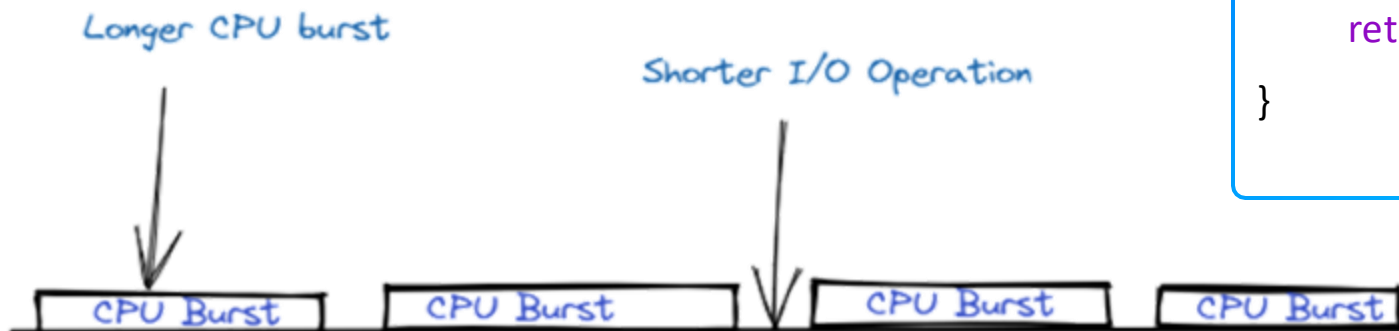
Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



1. Các khái niệm cơ bản về định thời

Tiến trình hướng CPU (CPU-bound)

- Tiến trình yêu cầu thời gian thực thi trên CPU nhiều.
- Thời gian hoàn thành chương trình phụ thuộc vào tốc độ thực thi của CPU.



```
#include <stdio.h>

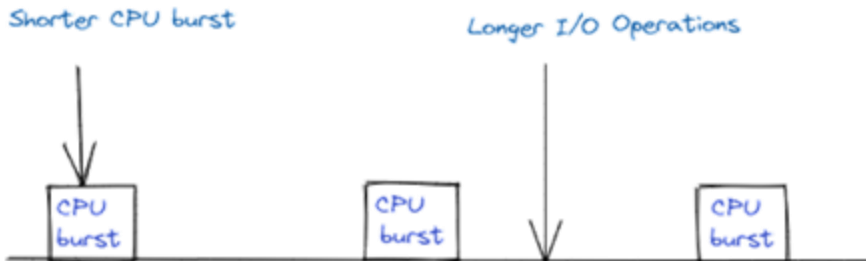
int main() {
    long long start = 1, end = 1000000,
    total = 0;
    for (long long i = start; i <= end; i++){
        total += i;
    }
    printf("Sum of numbers from %lld to %lld is %lld\n",
    start, end, total);
    return 0;
}
```



1. Các khái niệm cơ bản về định thời

Tiến trình hướng I/O (I/O-bound)

- Tiến trình yêu cầu thời gian thực thi trên ngoại vi nhiều hơn.
- Thời gian hoàn thành chương trình phụ thuộc chu kỳ đợi cho các thao tác nhập/xuất.



```
#include <stdio.h>

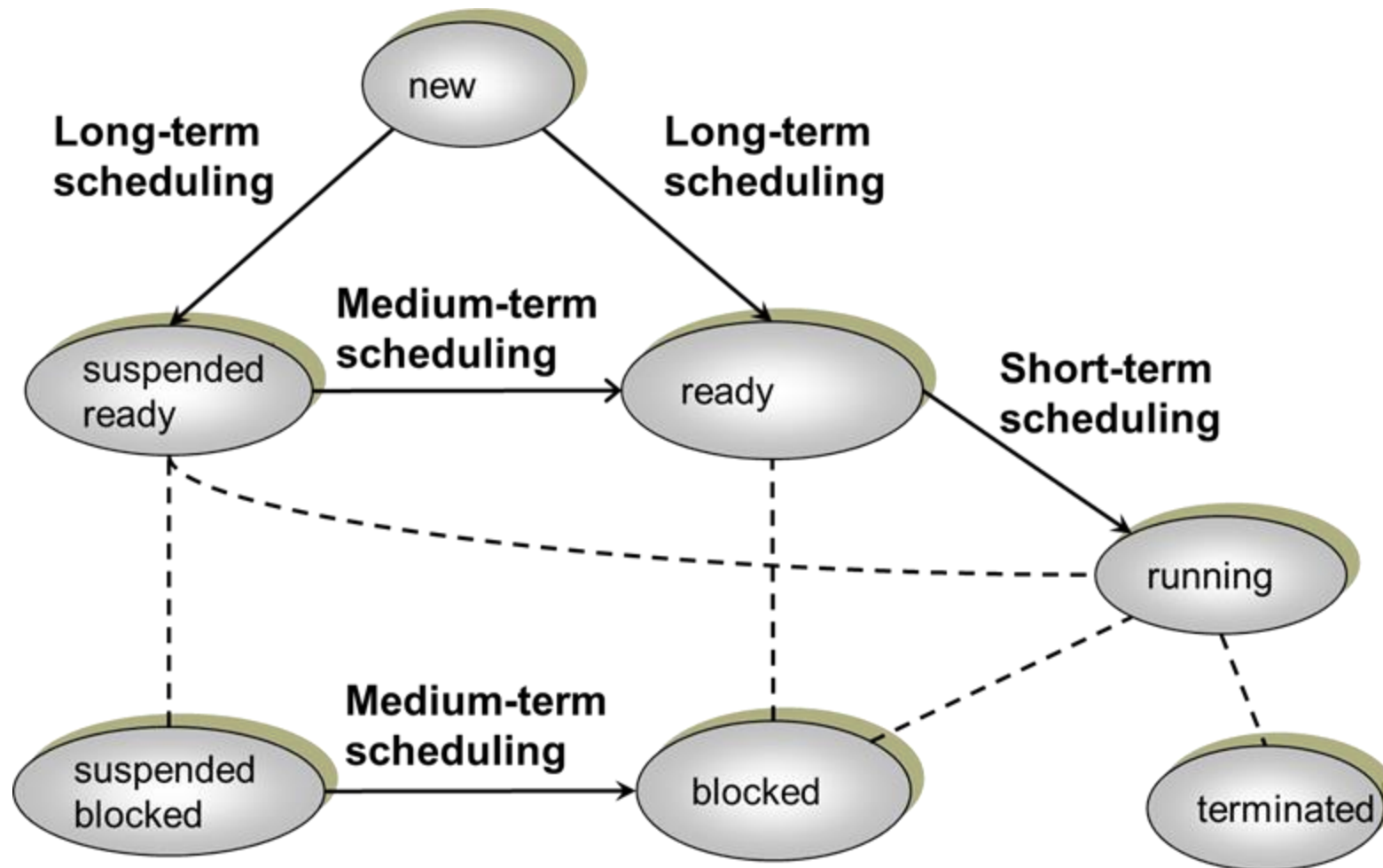
int main(){
    FILE *fp;
    char filename[] = "example.txt";
    int total = 0, ch;
    fp = fopen(filename, "r");
    if (fp == NULL){
        printf("Failed to open file %s\n", filename);
        return 1;
    }
    while ((ch = fgetc(fp)) != EOF){
        total++;
    }
    fclose(fp);
    printf("Total number of characters in file %s is %d\n", filename,
    total);
    return 0;
}
```



CÁC LOẠI ĐỊNH THỜI

02.

2. Các loại định thời





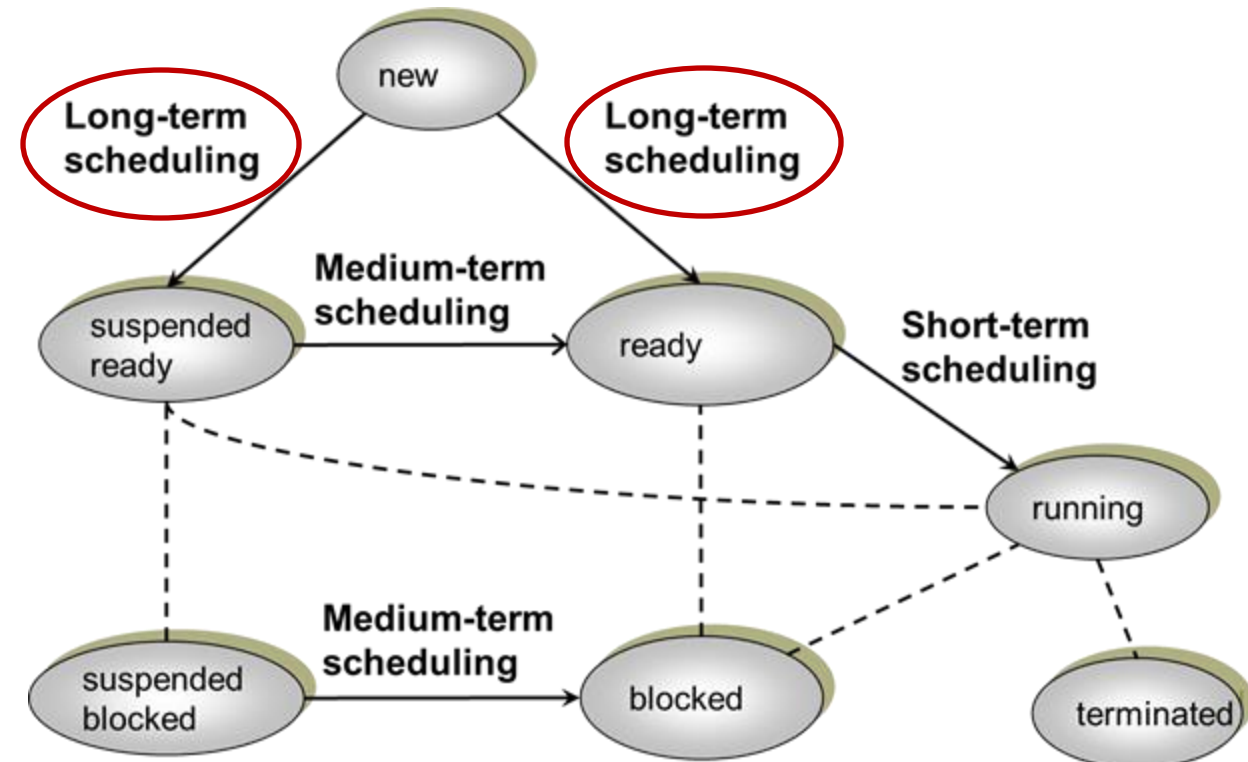
CÁC LOẠI ĐỊNH THỜI

2.1. Định thời dài (Long-term scheduling)

02.

2.1. Định thời dài (Long-term Scheduling)

- Xác định chương trình nào được chấp nhận nạp vào hệ thống để thực thi.
→ **Điều khiển mức độ đa chương của hệ thống.**
- Định thời dài thường cố gắng duy trì xen lẫn giữa tiến trình hướng CPU (CPU-bound process) và tiến trình hướng I/O (I/O-bound process).





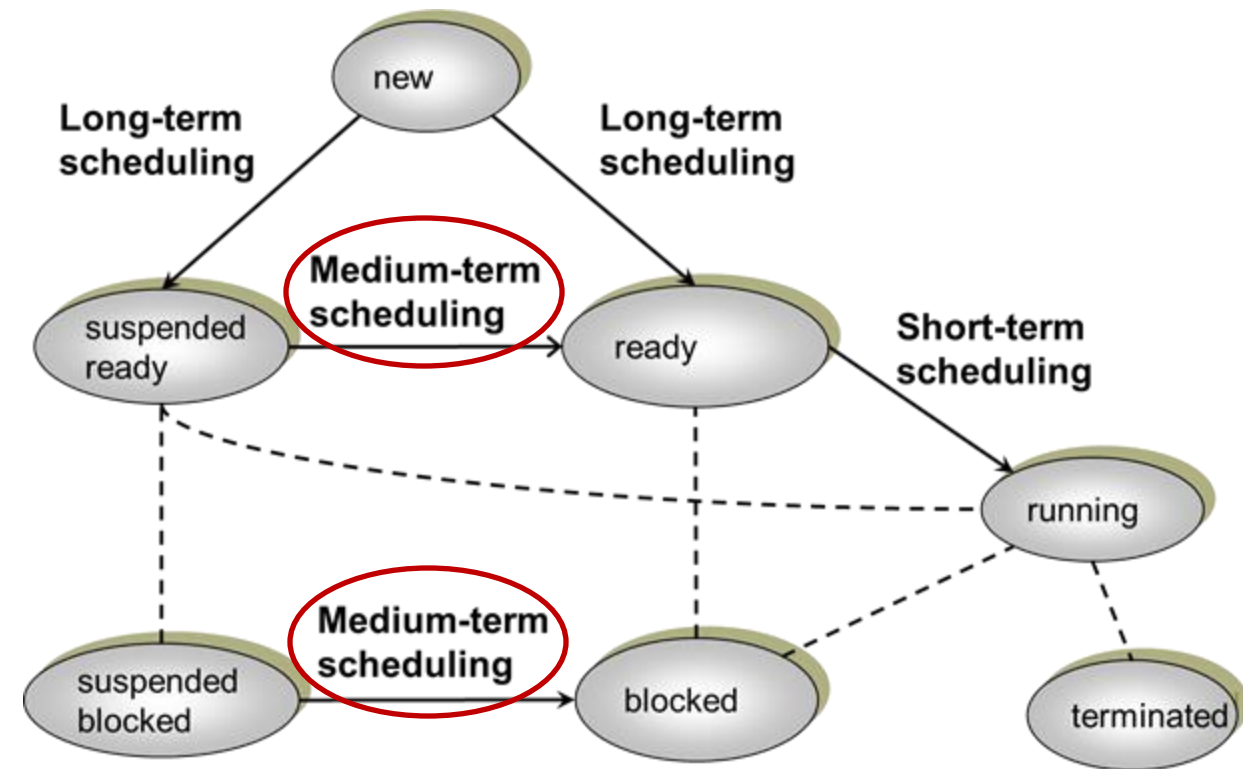
CÁC LOẠI ĐỊNH THỜI

2.2. Định thời vừa (Medium-term scheduling)

02.

2.2. Định thời vừa (Medium-term scheduling)

- Định thời vừa quyết định tiến trình nào được đưa vào (swap in) và đưa ra khỏi (swap out) bộ nhớ chính trong quá trình thực thi của hệ thống.
- Được thực hiện bởi thành phần quản lý bộ nhớ (và sẽ được thảo luận ở chương về quản lý bộ nhớ).





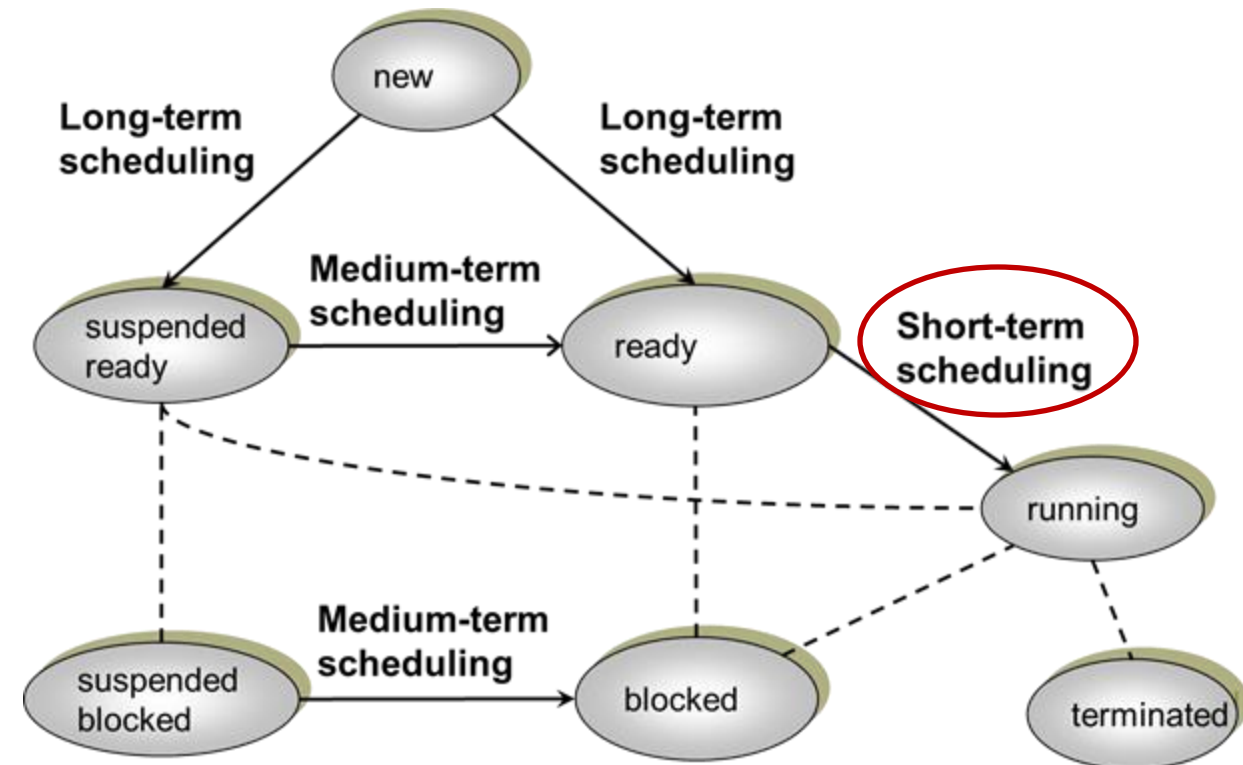
CÁC LOẠI ĐỊNH THỜI

2.3. Định thời ngắn (Short-term scheduling)

02.

2.3. Định thời ngắn (Short-term scheduling)

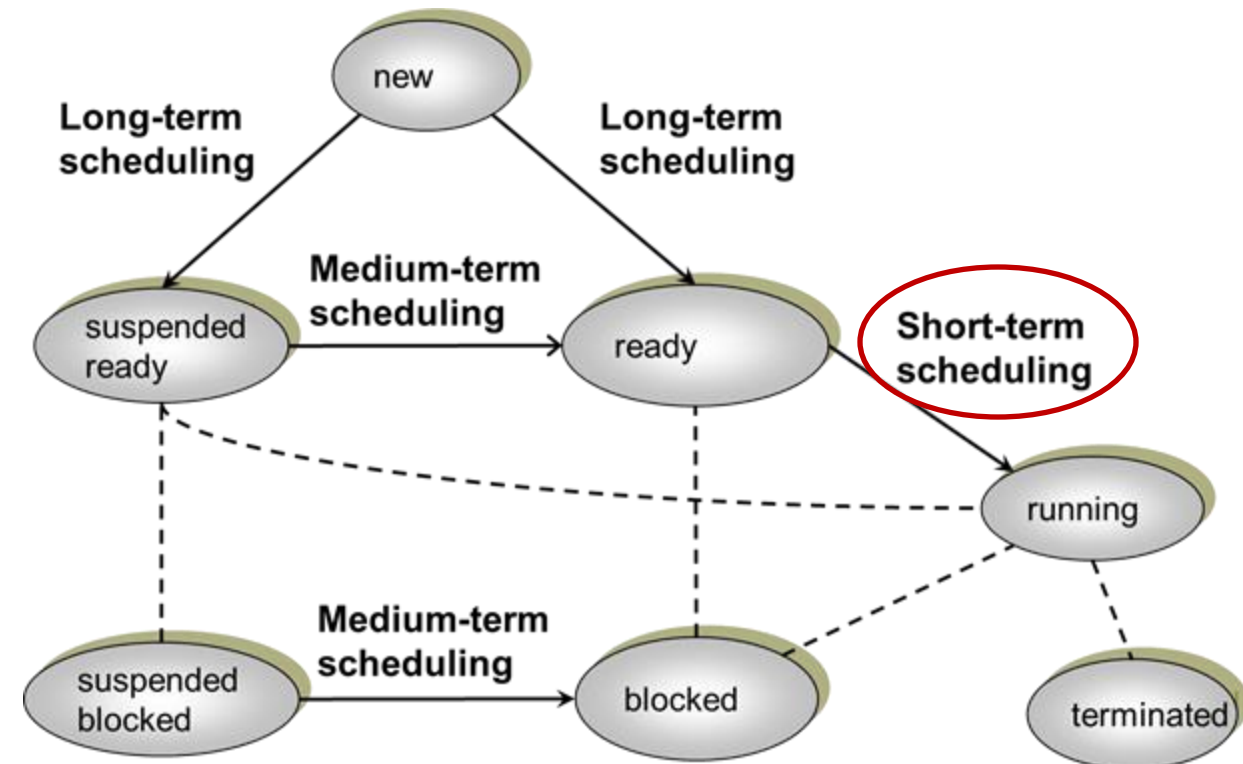
- Còn được gọi là **định thời CPU** (**CPU scheduling**).
- Xác định tiến trình nào trong ***hàng đợi sẵn sàng (ready queue)*** sẽ được chiếm CPU để thực thi kế tiếp.
- Đối với hệ thống hỗ trợ nhân đa luồng (multithreaded kernel), việc định thời CPU là do OS chọn ***kernel thread*** được chiếm CPU.



2.3. Định thời ngắn (Short-term scheduling)

- Bộ định thời ngắn được gọi khi có một trong các sự kiện/interrupt sau xảy ra:

- Ngắt thời gian (clock interrupt)
- Ngắt ngoại vi (I/O interrupt)
- Lời gọi hệ thống (operating system call)
- Tín hiệu đồng bộ hóa (Sẽ trao đổi sau ở Chương 5)





2.3. Định thời ngắn (Short-term scheduling)

Bộ định thời ngắn (Short-term scheduler)

- Bộ định thời sẽ chuyển quyền điều khiển CPU về cho tiến trình được chọn.
- Quá trình chuyển đổi bao gồm:
 - Chuyển ngữ cảnh (sử dụng thông tin ngữ cảnh trong PCB).
 - Chuyển chế độ người dùng.
 - Nhảy đến vị trí thích hợp trong chương trình ứng dụng để khởi động lại chương trình (sử dụng thông tin địa chỉ tại program counter trong PCB).
- Công việc này gây ra phí tổn
 - **Dispatch latency**: thời gian mà bộ định thời dừng một tiến trình và khởi động một tiến trình khác.



CÁC TIÊU CHUẨN ĐỊNH THỜI CPU

03.



3. Các tiêu chuẩn định thời CPU

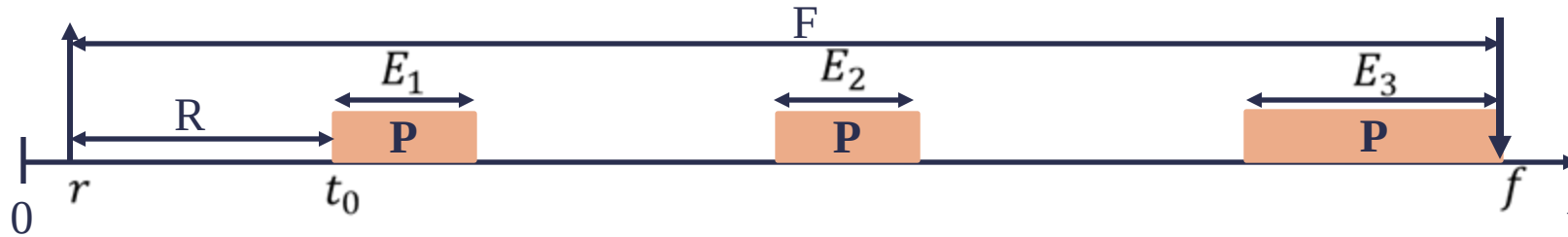
Hướng người dùng (user-oriented)

- **Thời gian đáp ứng** (*Response time*): khoảng thời gian từ lúc tiến trình gửi yêu cầu thực thi đến khi yêu cầu được đáp ứng lần đầu tiên (trong các hệ thống time-sharing, interactive system) → **cực tiểu**
- **Thời gian hoàn thành** (*Turnaround time*): khoảng thời gian từ lúc một tiến trình được nạp vào hệ thống đến khi tiến trình đó kết thúc → **cực tiểu**
- **Thời gian đợi** (*Waiting time*): tổng thời gian một tiến trình đợi trong ready queue → **cực tiểu**



3. Các tiêu chuẩn định thời CPU

Cách xác định các thông số định thời



Giả sử :

- Quá trình thực thi một tiến trình P gồm nhiều phần.
- r là thời điểm xuất hiện của P trong hệ thống (Arrival Time/Release Time).
- t_0 là thời điểm P được thực thi lần đầu tiên.
- f là thời điểm tiến trình P hoàn thành việc thực thi (Finishing Time).

Gọi R , F , và W lần lượt là thời gian đáp ứng, thời gian hoàn thành và thời gian đợi của tiến trình P. Khi đó:

$$R = t_0 - r$$

$$F = f - r$$

$$W = f - r - E$$

Trong đó E là thời gian yêu cầu của P để thực thi trên CPU (hay CPU Burst)

$$E = E_1 + E_2 + E_3$$



3. Các tiêu chuẩn định thời CPU

Hướng hệ thống (System oriented)

- **Hiệu năng sử dụng CPU** (*processor utilization*): định thời sao cho CPU càng bận càng tốt → **cực đại**
- **Tính công bằng** (*fairness*): tất cả tiến trình phải được **đối xử như nhau**.
- **Thông lượng** (*throughput*): số tiến trình hoàn tất công việc trong một đơn vị thời gian → **cực đại**



CÁC GIẢI THUẬT ĐỊNH THỜI

4.1. Giải thuật định thời

04.



4.1. Giải thuật định thời

Một giải thuật định thời thông thường bao gồm hai yếu tố:

- **Hàm chọn lựa** (*selection function*): mô tả cách thức (căn cứ) để chọn tiến trình nào trong *ready queue* được thực thi (Các hàm chọn lựa thường được xây dựng dựa trên độ ưu tiên, yêu cầu về tài nguyên, đặc điểm thực thi của tiến trình,...).
- **Chế độ quyết định** (*decision mode*): quyết định thời điểm thực hiện hàm chọn lựa để định thời.



4.1. Giải thuật định thời

Các chế độ quyết định

Có hai chế độ quyết định thường được áp dụng:

- **Không trưng dụng (Non-preemptive)**

- Khi ở trạng thái running, tiến trình sẽ thực thi cho đến khi kết thúc hoặc bị ngắt (blocked) do yêu cầu I/O.

- **Trưng dụng (Preemptive)**

- Tiến trình đang thực thi (ở trạng thái running) có thể bị ngắt giữa chừng và chuyển về trạng thái ready.
- Chi phí cao hơn chế độ không trưng dụng nhưng đánh đổi lại bằng thời gian đáp ứng tốt hơn vì không có trường hợp một tiến trình độc chiếm CPU quá lâu.



4.1. Giải thuật định thời

Thời điểm thực thi hàm chọn lựa

- Hàm chọn lựa được thực thi vào các thời điểm sau:

(1) Có tiến trình chuyển từ trạng thái **running** sang **waiting**.

(2) Có tiến trình chuyển từ trạng thái **running** sang **ready**.

(3) Có tiến trình chuyển từ trạng thái **waiting**, **new** sang **ready**.

(4) Kết thúc thực thi của một tiến trình.

→ (1) và (4) không cần lựa chọn loại định thời, (2) và (3) cần.

- Việc thực thi hàm chọn lựa trong trường hợp (1) và (4) không phụ thuộc vào loại giải thuật định thời và thường áp dụng chế độ không trưng dụng.
- Ngược lại, trường hợp (2) và (3) phụ thuộc vào loại giải thuật định thời và thường áp dụng chế độ trưng dụng.

→ Thực hiện theo cơ chế nào khó hơn?
Tại sao?



4.1. Giải thuật định thời

Các giải thuật định thời

- First-Come, First-Served (FCFS)
- Shortest Job First (SJF)
- Shortest Remaining Time First (SRTF)
- Round-Robin (RR)
- Priority Scheduling
- Highest Response Ratio Next (HRRN)
- Multilevel Queue
- Multilevel Feedback Queue



CÁC GIẢI THUẬT ĐỊNH THỜI

4.2. First-Come, First-Served (FCFS)

04.

4.2. First-Come, First-Served (FCFS)

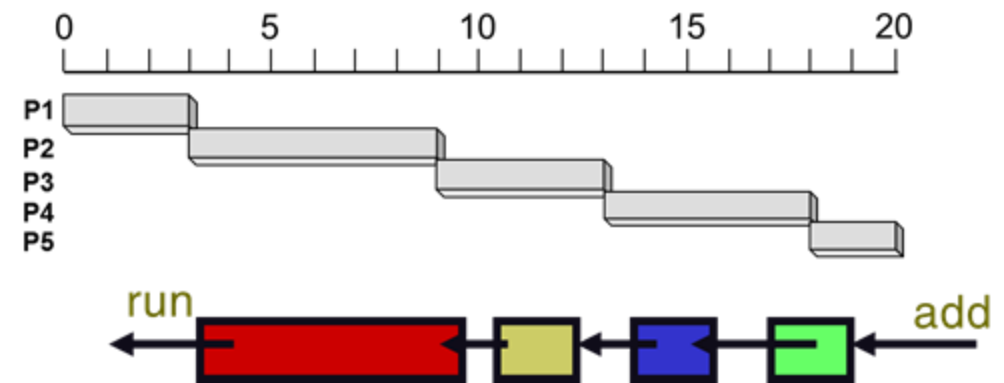
- **Hàm lựa chọn:**

- Tiến trình nào yêu cầu CPU trước sẽ được cấp phát CPU trước.
- Tiến trình sẽ thực thi đến khi kết thúc hoặc bị blocked do I/O.

- **Chế độ quyết định:** không trưng dụng (non-preemptive).

- **Hiện thực:** sử dụng hàng đợi FIFO (FIFO queues)

- Tiến trình mới xuất hiện được thêm vào cuối hàng đợi.
- Tiến trình được lựa chọn để xử lý được lấy từ đầu của hàng đợi.

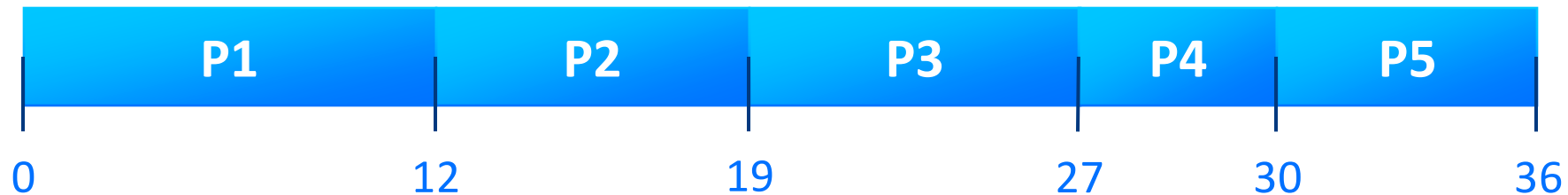




Ví dụ: First-Come, First-Server

Process	Arrival Time	Burst Time
P1	0	12
P2	2	7
P3	5	8
P4	9	3
P5	12	6

Giản đồ Gantt





Ví dụ: First-Come, First-Server

Process	Arrival Time	Burst Time
P1	0	12
P2	2	7
P3	5	8
P4	9	3
P5	12	6

Thời gian đáp ứng:

P1 = 0

$ART = (0+10+14+18+18)/5$

P2 = 10

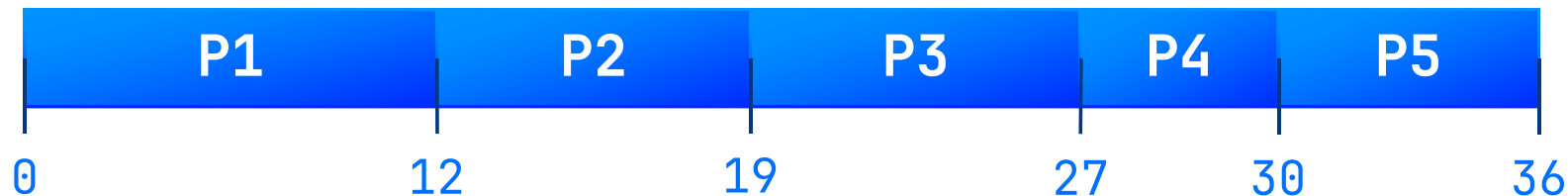
$= 12$

P3 = 14

P4 = 18

P5 = 18

Giản đồ Gantt





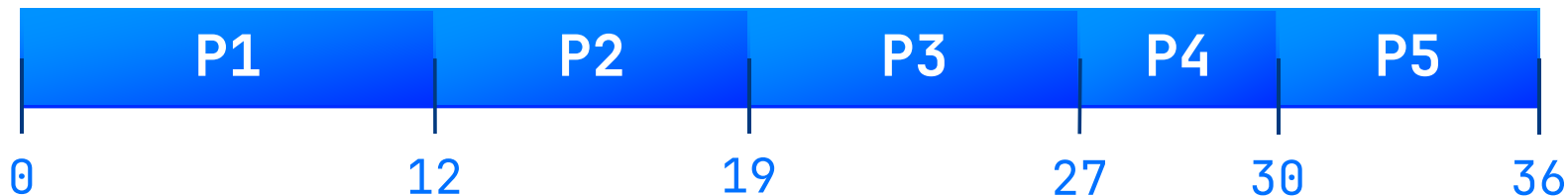
Ví dụ: First-Come, First-Server

Process	Arrival Time	Burst Time
P1	0	12
P2	2	7
P3	5	8
P4	9	3
P5	12	6

Thời gian hoàn thành:

P1 = 12 $ATaT = (12+17+22+21+24)/5$
P2 = 17 $= 19.2$
P3 = 22
P4 = 21
P5 = 24

Giản đồ Gantt





Ví dụ: First-Come, First-Server

Process	Arrival Time	Burst Time
P1	0	12
P2	2	7
P3	5	8
P4	9	3
P5	12	6

Thời gian chờ:

$$P1 = 12 - 12 = 0$$

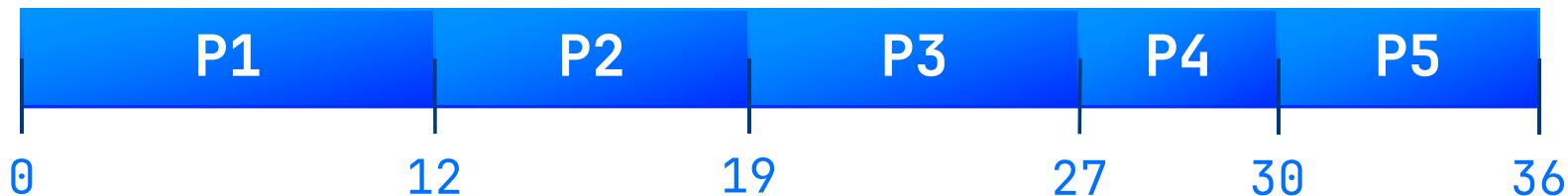
$$P2 = 17 - 7 = 10$$

$$P3 = 22 - 8 = 14$$

$$P4 = 21 - 3 = 18$$

$$P5 = 24 - 6 = 18$$

Giản đồ Gantt





Ví dụ: First-Come, First-Server

Process	Arrival Time	Burst Time
P1	0	12
P2	2	7
P3	5	8
P4	9	3
P5	12	6

Thời gian chờ:

P1 = 0

P2 = 10

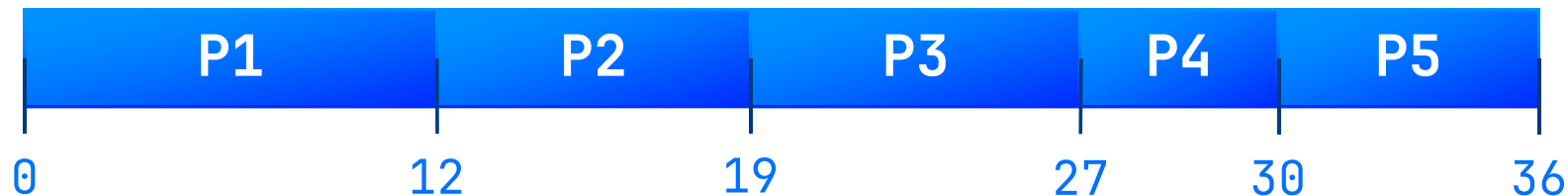
P3 = 14

P4 = 18

P5 = 18

$$AWT = (0 + 10 + 14 + 18 + 18) / 5 \\ = 12$$

Giản đồ Gantt





Bài tập: First-Come, First-Server

Bài tập

Process	Arrival Time	Burst Time
P1	4	9
P2	0	6
P3	16	10
P4	7	20
P5	6	4

Thực hiện định thời các tiến trình trên sử dụng giải thuật FCFS:

- Vẽ giản đồ Gantt
- Tính thời gian chờ trung bình, thời gian đáp ứng trung bình, thời gian hoàn thành trung bình



Bài tập: First-Come, First-Server

Bài tập

Process	Arrival Time	Burst Time
P1	4	9
P2	0	6
P3	16	10
P4	7	20
P5	6	4

Process	Arrival Time	Burst Time
P2	0	6
P1	4	9
P5	6	4
P4	7	20
P3	16	10

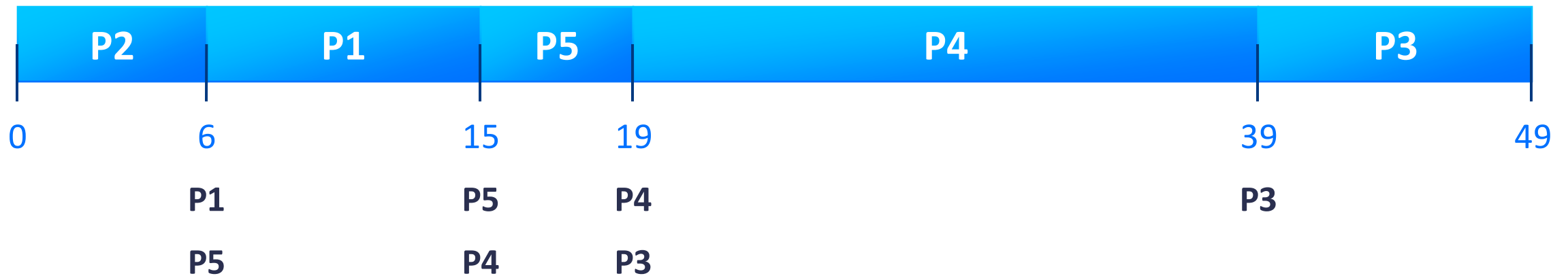


Sắp xếp theo thứ tự Arrival Time



Bài tập: First-Come, First-Server

Process	Arrival Time	Burst Time
P2	0	6
P1	4	9
P5	6	4
P4	7	20
P3	16	10





Bài tập: First-Come, First-Server

Process	Arrival Time	Burst Time
P2	0	6
P1	4	9
P5	6	4
P4	7	20
P3	16	10

Thời gian đáp ứng:

P1 = 2

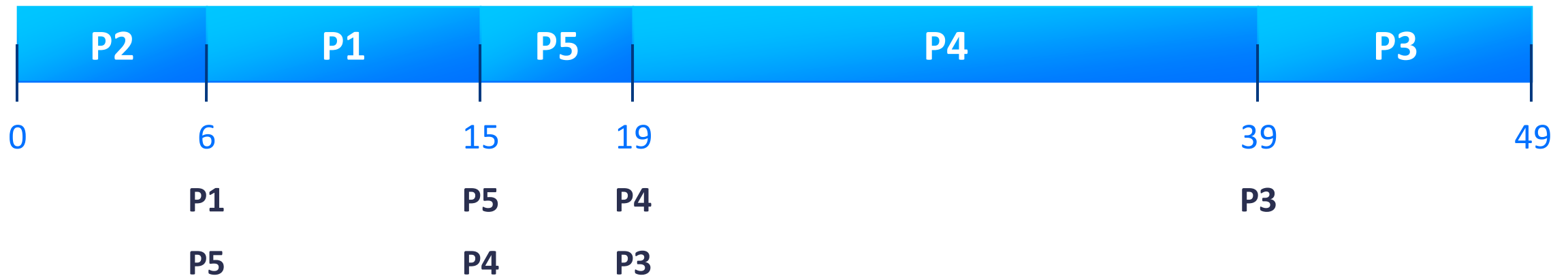
P2 = 0

P3 = 23

P4 = 12

P5 = 9

$$\begin{aligned} \text{ART} &= (2+0+23+12+9)/5 \\ &= 9.2 \end{aligned}$$





Bài tập: First-Come, First-Server

Process	Arrival Time	Burst Time
P2	0	6
P1	4	9
P5	6	4
P4	7	20
P3	16	10

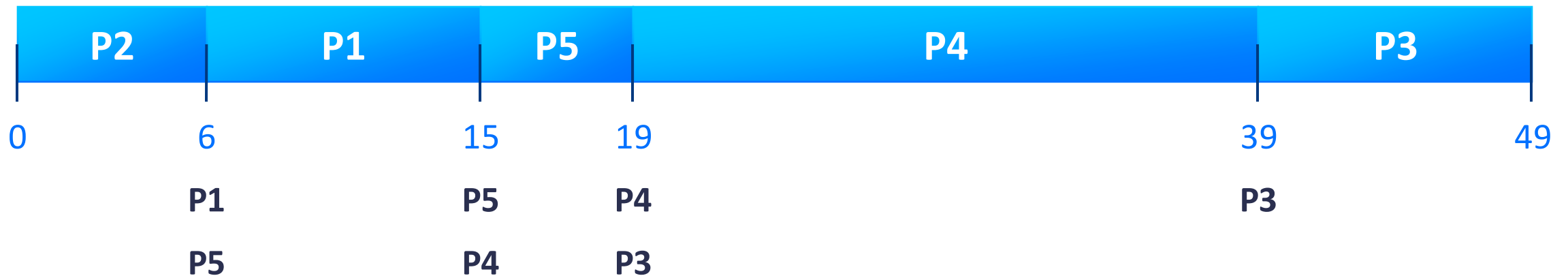
Thời gian hoàn thành:

$$\begin{aligned} P1 &= 11 & \text{ATaT} &= (11+6+33+32+13)/5 \\ P2 &= 6 & &= 19 \end{aligned}$$

$$P3 = 33$$

$$P4 = 32$$

$$P5 = 13$$





Bài tập: First-Come, First-Server

Process	Arrival Time	Burst Time
P2	0	6
P1	4	9
P5	6	4
P4	7	20
P3	16	10

Thời gian chờ:

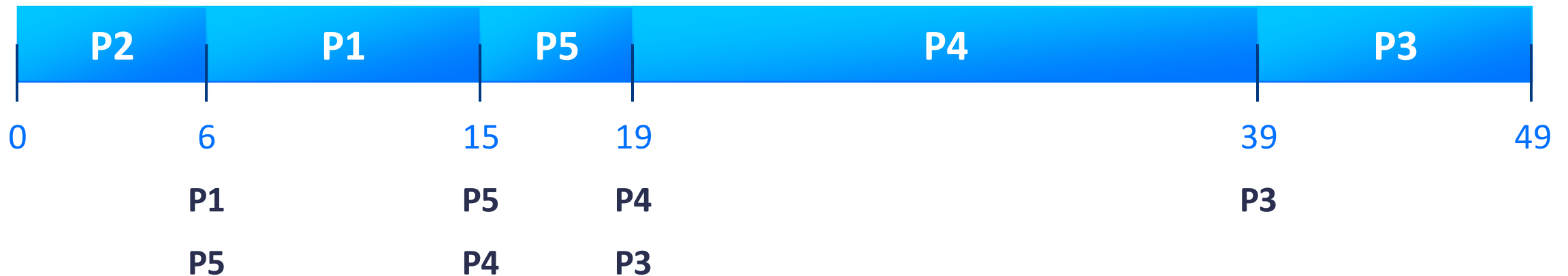
$$P1 = 11 - 9 = 2$$

$$P2 = 6 - 6 = 0$$

$$P3 = 33 - 10 = 23$$

$$P4 = 32 - 20 = 12$$

$$P5 = 13 - 4 = 9$$





Bài tập: First-Come, First-Server

Process	Arrival Time	Burst Time
P2	0	6
P1	4	9
P5	6	4
P4	7	20
P3	16	10

Thời gian chờ:

P1 = 2

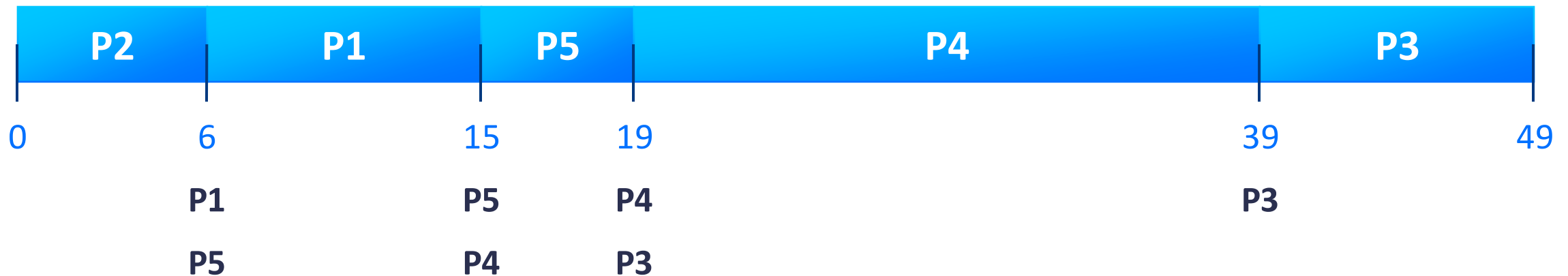
P2 = 0

P3 = 23

P4 = 12

P5 = 9

$$\begin{aligned} \text{AWT} &= (2+0+23+12+9)/5 \\ &= 9.2 \end{aligned}$$





THẢO LUẬN

